Assignment 10

1. Create an Ubuntu EC2 instance using Terraform. Configure a web server and display your PRN number and name on the home page.

```
prajwal@prajwal-ubuntu:~$ mkdir ~/terraform_assignment && cd ~/terraform_assignment
prajwal@prajwal-ubuntu:~/terraform_assignment$ nano main.tf
prajwal@prajwal-ubuntu:~/terraform_assignment$
```

```
  GNU nano 8.7.1                                                main.tf *
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1" # Replace with your target AWS Region
}

# 1. Fetch latest Ubuntu 22.04 LTS AMI
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical ID

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

# 2. Security Group allowing HTTP and SSH traffic
resource "aws_security_group" "web_sg" {
  name        = "webserver-sg"
  description = "Allow HTTP and SSH inbound traffic"
```

```
  GNU nano 8.7.1                                                main.tf *
  ingress {
    description = "HTTP access"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  ingress {
    description = "SSH access"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

# 3. Provision EC2 Instance & Web Server
resource "aws_instance" "web_server" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t2.micro"
  vpc_security_group_ids = [aws_security_group.web_sg.id]

  # User data script installs Nginx and creates the home page
  user_data = <<-EOF
```

```
 GNU nano 8.7.1                                                  main.tf *
  # User data script installs Nginx and creates the home page
  user_data = <<-EOF
              #!/bin/bash
              apt-get update -y
              apt-get install -y nginx
              systemctl start nginx
              systemctl enable nginx

              # Replace with your actual Name and PRN Number
              cat <<HTML > /var/www/html/index.html
              <!DOCTYPE html>
              <html>
              <head><title>Student Profile</title></head>
              <body>
                  <h1>Student Information</h1>
                  <p><b>Name:</b> Prajwal</p>
                  <p><b>PRN:</b> 260840123032</p>
              </body>
              </html>
              HTML
              EOF

  tags = {
    Name = "Ubuntu-WebServer"
  }
}

# 4. Output the public IP address
output "web_server_public_ip" {
  value       = aws_instance.web_server.public_ip
  description = "Public IP of the web server"
}
```

```
prajwal@prajwal-ubuntu:~/terraform_assignment$ terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
prajwal@prajwal-ubuntu:~/terraform_assignment$ terraform plan
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 0s [id=ami-082a1ff88c78c6290]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

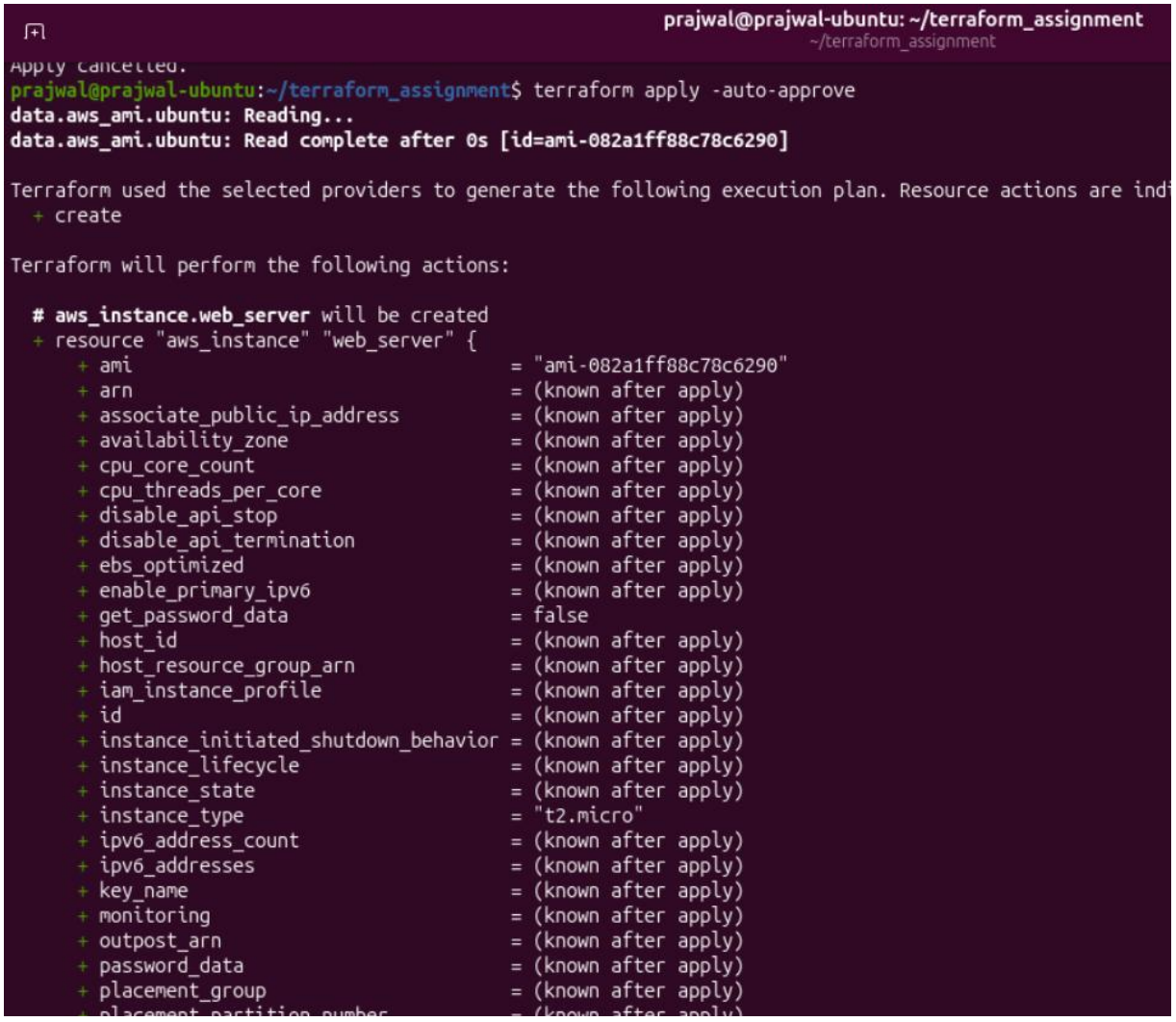
Terraform will perform the following actions:
```

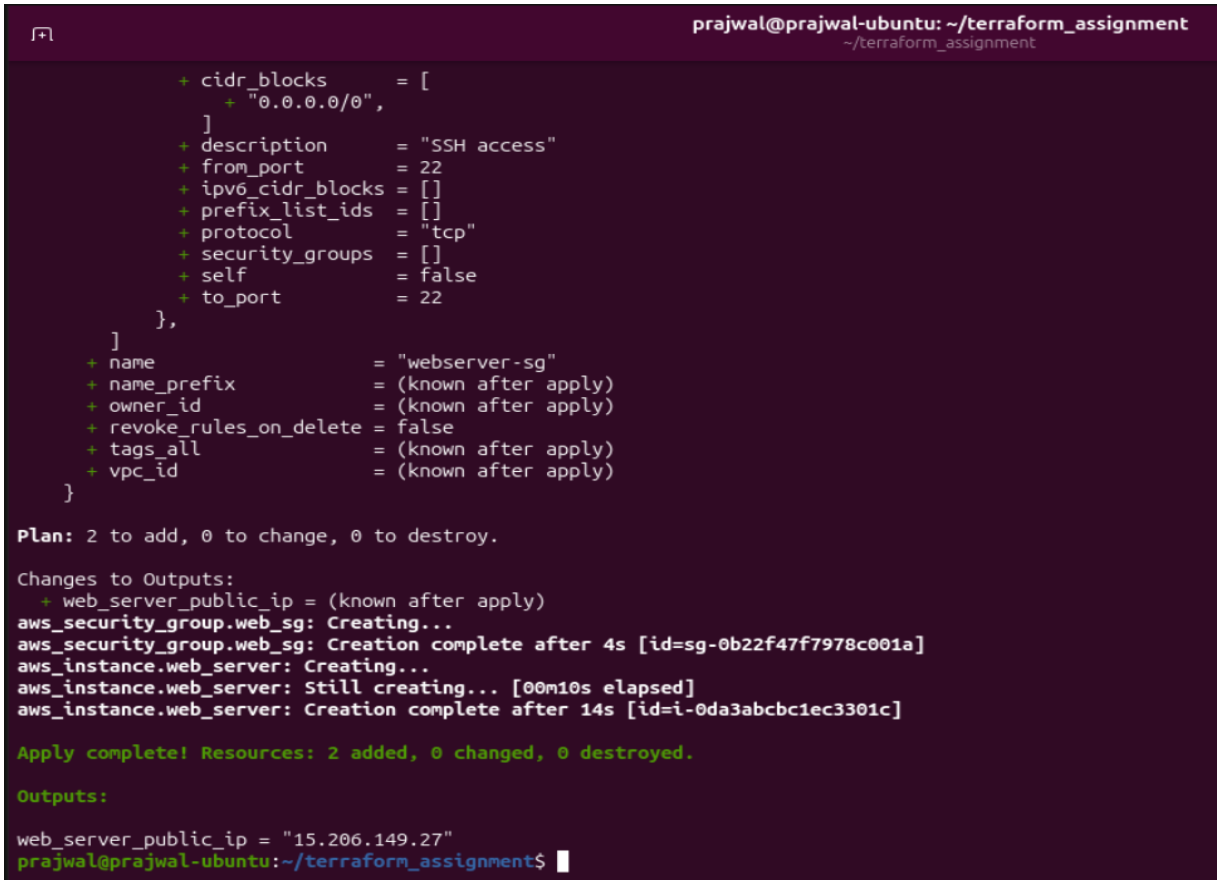
```
  # aws_instance.web_server will be created
  + resource "aws_instance" "web_server" {
      + ami                                  = "ami-082a1ff88c78c6290"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t2.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = (known after apply)
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
      + password_data                        = (known after apply)
      + placement_group                      = (known after apply)
      + placement_partition_number           = (known after apply)
      + primary_network_interface_id         = (known after apply)
      + private_dns                          = (known after apply)
      + private_ip                           = (known after apply)
      + public_dns                           = (known after apply)
      + public_ip                            = (known after apply)
      + secondary_private_ips                = (known after apply)
      + security_groups                      = (known after apply)
      + source_dest_check                    = true
      + spot_instance_request_id             = (known after apply)
```

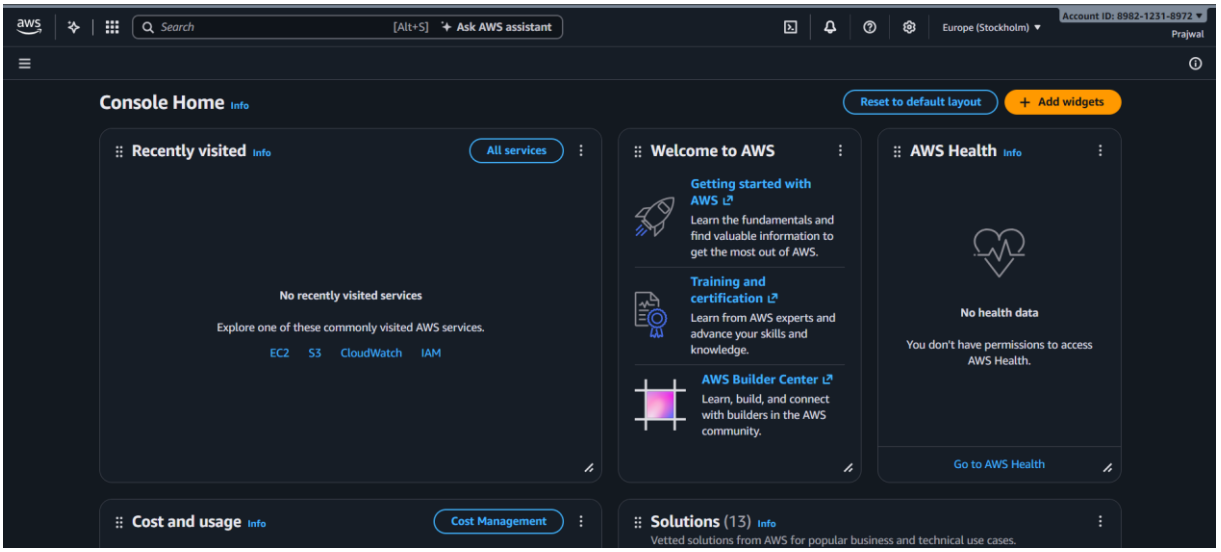
```
                                                                prajwal@prajwal
                                                                ~/
              + description      = "HTTP access"
              + from_port        = 80
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "tcp"
              + security_groups  = []
              + self             = false
              + to_port          = 80
            },
          + {
              + cidr_blocks      = [
                  + "0.0.0.0/0",
                ]
              + description      = "SSH access"
              + from_port        = 22
              + ipv6_cidr_blocks = []
              + prefix_list_ids  = []
              + protocol         = "tcp"
              + security_groups  = []
              + self             = false
              + to_port          = 22
            },
        ]
      + name                   = "webserver-sg"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + revoke_rules_on_delete = false
      + tags_all               = (known after apply)
      + vpc_id                 = (known after apply)
    }

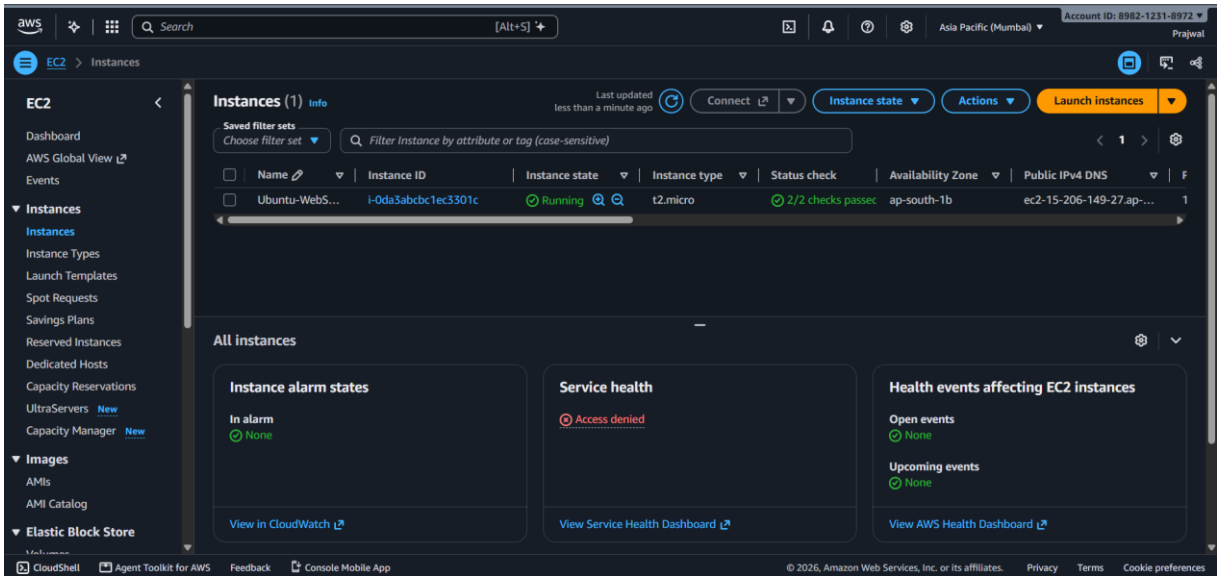
Plan: 2 to add, 0 to change, 0 to destroy.

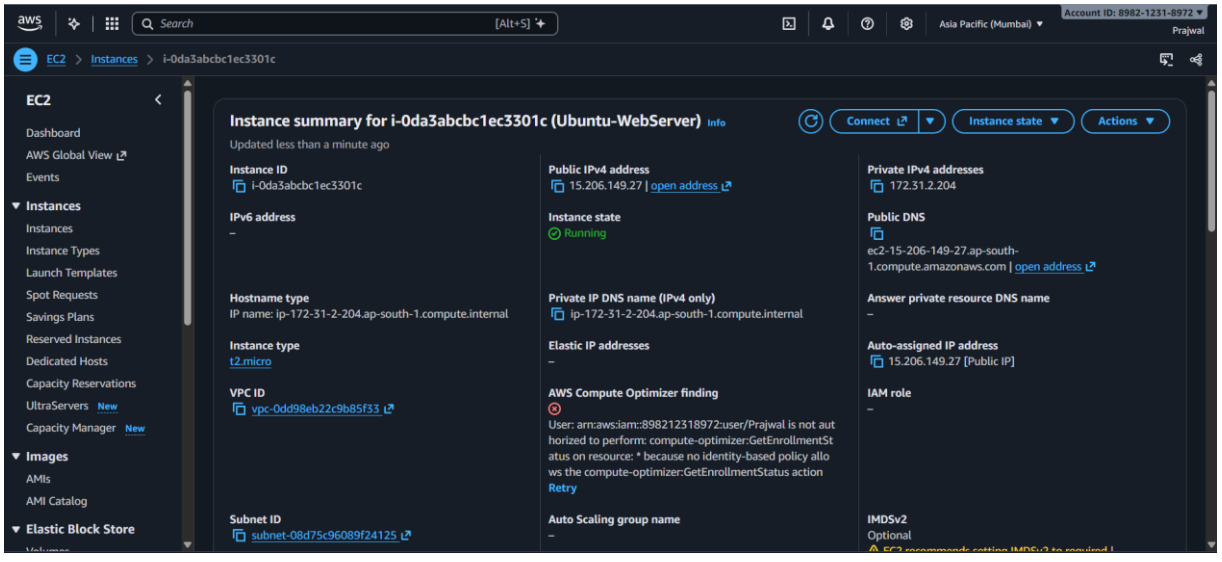
Changes to Outputs:
  + web_server_public_ip = (known after apply)
```

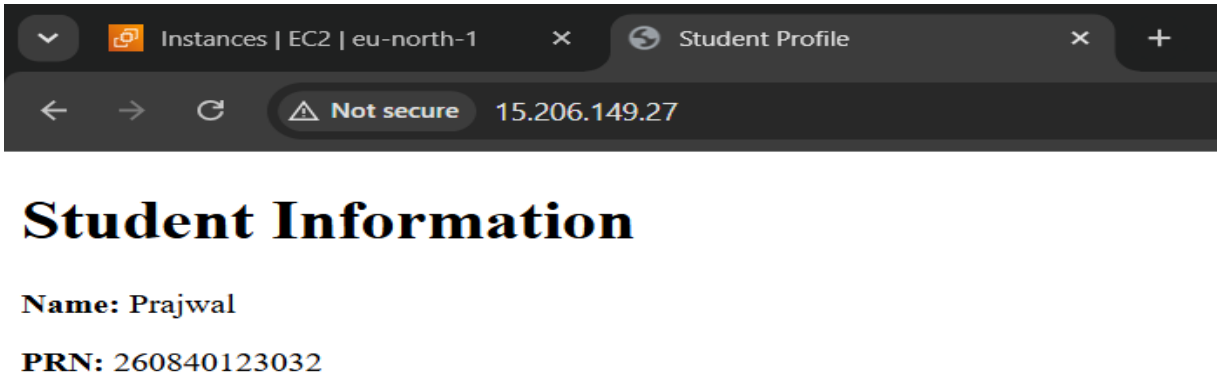

prajwal@prajwal-ubuntu: ~/terraform_assignment
~/terraform_assignment
Apply cancelled.
prajwal@prajwal-ubuntu:~/terraform_assignment$ terraform apply -auto-approve
data.aws_ami.ubuntu: Reading...
data.aws_ami.ubuntu: Read complete after 0s [id=ami-082a1ff88c78c6290]
Terraform used the selected providers to generate the following execution plan. Resource actions are ind
+ create
Terraform will perform the following actions:
aws_instance.web_server will be created
+ resource "aws_instance" "web_server" {
+ ami = "ami-082a1ff88c78c6290"
+ arn = (known after apply)
+ associate_public_ip_address = (known after apply)
+ availability_zone = (known after apply)
+ cpu_core_count = (known after apply)
+ cpu_threads_per_core = (known after apply)
+ disable_api_stop = (known after apply)
+ disable_api_termination = (known after apply)
+ ebs_optimized = (known after apply)
+ enable_primary_ipv6 = (known after apply)
+ get_password_data = false
+ host_id = (known after apply)
+ host_resource_group_arn = (known after apply)
+ iam_instance_profile = (known after apply)
+ id = (known after apply)
+ instance_initiated_shutdown_behavior = (known after apply)
+ instance_lifecycle = (known after apply)
+ instance_state = (known after apply)
+ instance_type = "t2.micro"
+ ipv6_address_count = (known after apply)
+ ipv6_addresses = (known after apply)
+ key_name = (known after apply)
+ monitoring = (known after apply)
+ outpost_arn = (known after apply)
+ password_data = (known after apply)
+ placement_group = (known after apply)


prajwal@prajwal-ubuntu: ~/terraform_assignment
~/terraform_assignment
+ cidr_blocks = [
+ "0.0.0.0/0",
]
+ description = "SSH access"
+ from_port = 22
+ ipv6_cidr_blocks = []
+ prefix_list_ids = []
+ protocol = "tcp"
+ security_groups = []
+ self = false
+ to_port = 22
},
]
+ name = "webserver-sg"
+ name_prefix = (known after apply)
+ owner_id = (known after apply)
+ revoke_rules_on_delete = false
+ tags_all = (known after apply)
+ vpc_id = (known after apply)
}
Plan: 2 to add, 0 to change, 0 to destroy.
Changes to Outputs:
+ web_server_public_ip = (known after apply)
aws_security_group.web_sg: Creating...
aws_security_group.web_sg: Creation complete after 4s [id=sg-0b22f47f7978c001a]
aws_instance.web_server: Creating...
aws_instance.web_server: Still creating... [00m10s elapsed]
aws_instance.web_server: Creation complete after 14s [id=i-0da3abcbc1ec3301c]
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
Outputs:
web_server_public_ip = "15.206.149.27"
prajwal@prajwal-ubuntu:~/terraform_assignment$


Search [Alt+S] Ask AWS assistant
Europe (Stockholm)
Account ID: 8982-1231-8972
Prajwal
Console Home Info
Reset to default layout
+ Add widgets
Recently visited Info
All services
No recently visited services
Explore one of these commonly visited AWS services.
EC2 S3 CloudWatch IAM
Welcome to AWS
Getting started with AWS
Learn the fundamentals and find valuable information to get the most out of AWS.
Training and certification
Learn from AWS experts and advance your skills and knowledge.
AWS Builder Center
Learn, build, and connect with builders in the AWS community.
AWS Health Info
No health data
You don't have permissions to access AWS Health.
Go to AWS Health
Cost and usage Info
Cost Management
Solutions (13) Info
Vetted solutions from AWS for popular business and technical use cases.


Search [Alt+S]
Asia Pacific (Mumbai)
Account ID: 8982-1231-8972
Prajwal
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Instances (1) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Filter Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
Ubuntu-WebS...
i-0da3abcbc1ec3301c
Running
t2.micro
2/2 checks passed
ap-south-1b
ec2-15-206-149-27.ap-...
All instances
Instance alarm states
In alarm
None
View in CloudWatch
Service health
Access denied
View Service Health Dashboard
Health events affecting EC2 instances
Open events
None
Upcoming events
None
View AWS Health Dashboard
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Search [Alt+S]
Asia Pacific (Mumbai)
Account ID: 8982-1231-8972
Prajwal
EC2 > Instances > i-0da3abcbc1ec3301c
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
UltraServers New
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Instance summary for i-0da3abcbc1ec3301c (Ubuntu-WebServer) Info
Updated less than a minute ago
Connect
Instance state
Actions
Instance ID
i-0da3abcbc1ec3301c
IPv6 address
–
Hostname type
IP name: ip-172-31-2-204.ap-south-1.compute.internal
Instance type
t2.micro
VPC ID
vpc-0dd98eb22c9b85f33
Subnet ID
subnet-08d75c96089f24125
Public IPv4 address
15.206.149.27 | open address
Instance state
Running
Private IP DNS name (IPv4 only)
ip-172-31-2-204.ap-south-1.compute.internal
Elastic IP addresses
–
AWS Compute Optimizer finding
User: arn:aws:iam::898212318972:user/Prajwal is not authorized to perform: compute-optimizer:GetEnrollmentStatus on resource: * because no identity-based policy allows the compute-optimizer:GetEnrollmentStatus action
Retry
Auto Scaling group name
–
Private IPv4 addresses
172.31.2.204
Public DNS
ec2-15-206-149-27.ap-south-1.compute.amazonaws.com | open address
Answer private resource DNS name
–
Auto-assigned IP address
15.206.149.27 [Public IP]
IAM role
–
IMDSv2
Optional



2. Using Terraform, create a new IAM user named `cdac` and delete/drop an existing IAM user named `ditiss`.

```
prajwal@prajwal-ubuntu:~/terraform_assignment$ mkdir question2 && cd question2
prajwal@prajwal-ubuntu:~/terraform_assignment/question2$ nano iam.tf
prajwal@prajwal-ubuntu:~/terraform_assignment/question2$
```

```
  GNU nano 8.7.1
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1" # Mumbai Region
}

# 1. Create a new IAM user named 'cdac'
resource "aws_iam_user" "cdac_user" {
  name = "cdac"

  tags = {
    Environment = "Development"
  }
}
```

```
prajwal@prajwal-ubuntu:~/terraform_assignment/question2$ terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

```
prajwal@prajwal-ubuntu:~/terraform_assignment/question2$ terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_iam_user.cdac_user will be created
  + resource "aws_iam_user" "cdac_user" {
      + arn           = (known after apply)
      + force_destroy = false
      + id            = (known after apply)
      + name          = "cdac"
      + path          = "/"
      + tags          = {
          + "Environment" = "Development"
        }
      + tags_all      = {
          + "Environment" = "Development"
        }
      + unique_id     = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

─────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
```

```
prajwal@prajwal-ubuntu:~/terraform_assignment/question2$ terraform apply -auto-approve

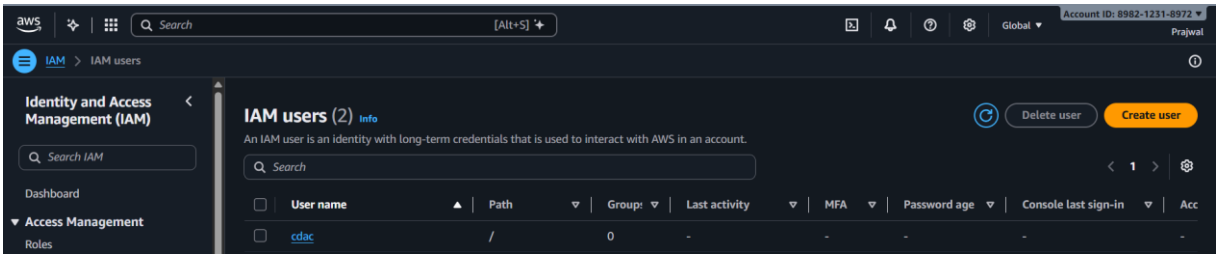
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

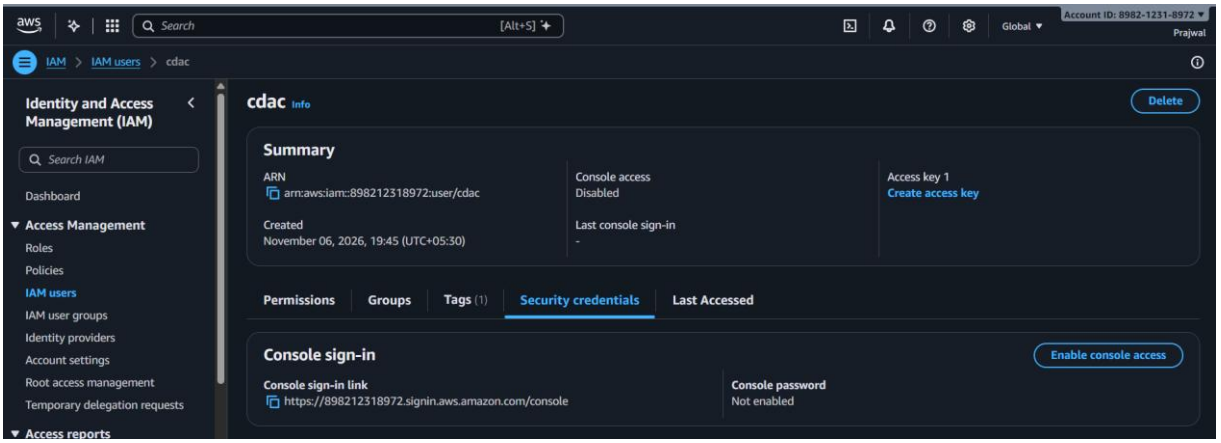
Terraform will perform the following actions:

  # aws_iam_user.cdac_user will be created
  + resource "aws_iam_user" "cdac_user" {
      + arn           = (known after apply)
      + force_destroy = false
      + id            = (known after apply)
      + name          = "cdac"
      + path          = "/"
      + tags          = {
          + "Environment" = "Development"
        }
      + tags_all      = {
          + "Environment" = "Development"
        }
      + unique_id     = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
aws_iam_user.cdac_user: Creating...
aws_iam_user.cdac_user: Creation complete after 3s [id=cdac]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
prajwal@prajwal-ubuntu:~/terraform_assignment/question2$
```





```
prajwal@prajwal-ubuntu:~/terraform_assignment/question2$ aws iam list-users --query "Users[*].UserName" --output table
------------
| ListUsers |
+-----------+
|  cdac     |
|  Prajwal  |
+-----------+
prajwal@prajwal-ubuntu:~/terraform_assignment/question2$ aws iam delete-user --user-name cdac
prajwal@prajwal-ubuntu:~/terraform_assignment/question2$ aws iam list-users --query "Users[*].UserName" --output table
------------
| ListUsers |
+-----------+
|  Prajwal  |
+-----------+
```

3. Configure Ansible in client-server mode. From the Ansible server, execute a shell script on the client machine that displays the message:

```
Hello DITISS
```

```
ansible-user@ansible-user-VMware-Virtual-Platform: ~

ansible-user@ansible-user-VMware-Virtual-Platform:~$ sudo nano /tmp/display_msg.sh
[sudo: authenticate] Password:
ansible-user@ansible-user-VMware-Virtual-Platform:~$ sudo chmod +x /tmp/display_msg.sh
ansible-user@ansible-user-VMware-Virtual-Platform:~$
```

```
ansible-user@ansible-user-VMware-Virtual-Platform:~$ ip -br -4 -c a
lo               UNKNOWN        127.0.0.1/8
ens33            UP             192.168.201.133/24
ansible-user@ansible-user-VMware-Virtual-Platform:~$ ^C
ansible-user@ansible-user-VMware-Virtual-Platform:~$ sudo apt install -y openssh-server
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-30          linux-image-unsigned-7.0.0-30-generic    linux-modules-7.0.0-30-generic  linux-tools-7
.0.0-30-generic
  linux-headers-7.0.0-30-generic  linux-main-modules-zfs-7.0.0-30-generic  linux-tools-7.0.0-30
Use 'sudo apt autoremove' to remove them.

Installing:
  openssh-server
```

```
ansible-user@ansible-user-VMware-Virtual-Platform:~$ sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
ansible-user@ansible-user-VMware-Virtual-Platform:~$ sudo ufw reload
Firewall not enabled (skipping reload)
```

```
ansible-user@ansible-user-VMware-Virtual-Platform:~$ cat << 'EOF' | sudo tee /tmp/display_msg.ssh
> echo "Hello DITISS"
> EOF
echo "Hello DITISS"
ansible-user@ansible-user-VMware-Virtual-Platform:~$ sudo chmod +x /tmp/display_msg.sh
ansible-user@ansible-user-VMware-Virtual-Platform:~$ whoami
ansible-user
```

```
prajwal@prajwal-ubuntu:~/terraform_assignment/question3$ ssh-copy-id ansible-user@192.168.201.133
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: ssh-add -L
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
ansible-user@192.168.201.133's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'ansible-user@192.168.201.133'"
and check to make sure that only the key(s) you wanted were added.

prajwal@prajwal-ubuntu:~/terraform_assignment/question3$ ssh ansible-user@192.168.201.133
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-43-generic x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

96 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status
```

```
ansible-user@ansible-user-VMware-Virtual-Platform:~$ mkdir -p ~/ansible_assignment && cd ~/ansible_assignment
ansible-user@ansible-user-VMware-Virtual-Platform:~/ansible_assignment$ nano inventory.ini
ansible-user@ansible-user-VMware-Virtual-Platform:~/ansible_assignment$
```

```
  GNU nano 8.7.1                                                inventory.ini *
[clients]
client1 ansible_host=192.168.201.133 ansible_user=ansible-user ansible_ssh_private_key_file=~/.ssh/id_rsa
```

```
ansible-user@ansible-user-VMware-Virtual-Platform:~/ansible_assignment$ nano inventory.ini
ansible-user@ansible-user-VMware-Virtual-Platform:~/ansible_assignment$ nano run_script.yml
```

```
  GNU nano 8.7.1                                                run_script.yml
---
- name: Create and Execute Shell Script on Client
  hosts: clients
  become: no
  tasks:
    - name: Ensure /tmp/display_msg.sh exists with correct content
      ansible.builtin.copy:
        dest: /tmp/display_msg.sh
        mode: '0755'
        content: |
          #!/bin/bash
          echo "Hello DITISS"

    - name: Run script located at /tmp/display_msg.sh
      ansible.builtin.command: /tmp/display_msg.sh
      register: script_output

    - name: Print output message
      ansible.builtin.debug:
        msg: "{{ script_output.stdout }}"
```


ansible-user@ansible-user-VMware-Virtual-Platform:~/ansible_assignment$ nano inventory.ini
ansible-user@ansible-user-VMware-Virtual-Platform:~/ansible_assignment$ nano run_script.yml
ansible-user@ansible-user-VMware-Virtual-Platform:~/ansible_assignment$ ansible-playbook -i inventory.ini run_script.yml -K
BECOME password:
PLAY [Create and Execute Shell Script on Client] ****
TASK [Gathering Facts] ****
[WARNING]: Host 'client1' is using the discovered Python interpreter at '/usr/bin/python3.14', but future installation of another
a different interpreter to be discovered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discov
ok: [client1]
TASK [Ensure /tmp/display_msg.sh exists with correct content] ****
ok: [client1]
TASK [Run script located at /tmp/display_msg.sh] ****
changed: [client1]
TASK [Print output message] ****
ok: [client1] => {
"msg": "Hello DITISS"
}
PLAY RECAP ****
client1 : ok=4 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0